NP

October 6, 2026

[1]:
```
import numpy as np
import pandas as pd
```

NUMPY

[2]:
```
print(np.ones(3))
print(np.zeros(3))
print(np.random.random(3))
```

```
[1. 1. 1.]
[0. 0. 0.]
[0.21556508 0.95969598 0.50140903]
```

[3]:
```
data = np.array([1,2,3,4,5])
print(data)
print(data.max())
```

```
[1 2 3 4 5]
5
```

[4]:
```
print(data+np.ones(1))
print(data-np.ones(1))
print(data*np.zeros(1))
print(data/data)
```

```
[2. 3. 4. 5. 6.]
[0. 1. 2. 3. 4.]
[0. 0. 0. 0. 0.]
[1. 1. 1. 1. 1.]
```

[5]:
```
a = np.array([1,2,3])
b = np.array([[1,2,3],[3,4,5]])
print("Row Matrix : \n",a)
print("Row * Col Matrix : \n",b)
```

```
Row Matrix :
 [1 2 3]
Row * Col Matrix :
 [[1 2 3]
 [3 4 5]]
```

[6]:
```
print("Zeros ((3, 4)): \n", np.zeros((3, 4)), "\n")
print("Ones ((2, 3)): \n", np.ones((2, 3)), "\n")
print("Full ((2, 2), 7): \n", np.full((2, 2), 7), "\n")
print("Eye (3): \n", np.eye(3), "\n")
print("Arange (0, 10, 2): \n", np.arange(0, 10, 2), "\n")
print("Linspace (0, 1, 5): \n", np.linspace(0, 1, 5), "\n")
print("random.rand (3, 3): \n", np.random.rand(3, 3), "\n")
print("random.randn (3): \n", np.random.randn(3), "\n")
print("Empty ((2, 3)): \n", np.empty((2, 3)), "\n")
```

```
Zeros ((3, 4)):
 [[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]

Ones ((2, 3)):
 [[1. 1. 1.]
 [1. 1. 1.]]

Full ((2, 2), 7):
 [[7 7]
 [7 7]]

Eye (3):
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

Arange (0, 10, 2):
 [0 2 4 6 8]

Linspace (0, 1, 5):
 [0.   0.25 0.5  0.75 1.  ]

random.rand (3, 3):
 [[0.37053068 0.66783653 0.84751211]
 [0.5718999  0.17257023 0.81302063]
 [0.48391227 0.51556429 0.82092161]]

random.randn (3):
 [-0.70023261 -0.67516959  1.39922535]

Empty ((2, 3)):
 [[1. 1. 1.]
 [1. 1. 1.]]
```

[7]:
```
print(a.shape)
print(a.ndim)
print(a.size)
print(a.dtype)
print(a.itemsize)
print(a.nbytes)
```

```
(3,)
1
3
int64
8
24
```

[8]:
```
print("a[0] \n", a[0], "\n")
print("a[-1] \n", a[-1], "\n")
print("a[1:4] \n", a[1:4], "\n")
print("a[::2] \n", a[::2], "\n")
print("b[0, 1] \n", b[0, 1], "\n")
print("b[:, 0] \n", b[:, 0], "\n")
print("b[1, :] \n", b[1, :], "\n")
print("b[0:2, 1:3] \n", b[0:2, 1:3], "\n")
```

```
a[0]
 1

a[-1]
 3

a[1:4]
 [2 3]

a[::2]
 [1 3]

b[0, 1]
 2

b[:, 0]
 [1 3]

b[1, :]
 [3 4 5]

b[0:2, 1:3]
 [[2 3]
 [4 5]]
```

[9]:
```
print("a[a > 2] \n", a[a > 2], "\n")
print("a[np.array([0,2,1])] \n", a[np.array([0,2,1])], "\n")
print("np.where(a>5, a, 0) \n", np.where(a>5, a, 0), "\n")
```

```
a[a > 2]
 [3]

a[np.array([0,2,1])]
 [1 3 2]

np.where(a>5, a, 0)
 [0 0 0]
```

[10]:
```
'''n = np.array(['A','B','C','D','A'])
m = np.random.rand(3,4)

m[n = 'A']
'''
```

[10]:
```
"n = np.array(['A','B','C','D','A'])\nm = np.random.rand(3,4)\n\nm[n = 'A']\n"
```

[11]:
```
print("a : ", a)
print("b : ", b)
print()
print("a + b \n", a + b, "\n")
print("a - b \n", a - b, "\n")
print("a * b \n", a * b, "\n")
print("a / b \n", a / b, "\n")
print("a ** 2 \n", a ** 2, "\n")
print("a % 3 \n", a % 3, "\n")
```

```
a :  [1 2 3]
b :  [[1 2 3]
 [3 4 5]]

a + b
 [[2 4 6]
 [4 6 8]]

a - b
 [[ 0  0  0]
 [-2 -2 -2]]

a * b
 [[ 1  4  9]
 [ 3  8 15]]
```

```
a / b
 [[1.         1.         1.        ]
 [0.33333333 0.5        0.6       ]]

a ** 2
 [1 4 9]

a % 3
 [1 2 0]
```

[12]:
```
print("a : ", a, "\n")
print("np.sqrt(a) \n", np.sqrt(a), "\n")
print("np.exp(a) \n", np.exp(a), "\n")
print("np.log(a) \n", np.log(a), "\n")
print("np.sin(a) \n", np.sin(a), "\n")
print("np.cos(a) \n", np.cos(a), "\n")
print("np.abs(a) \n", np.abs(a), "\n")
```

```
a :  [1 2 3]

np.sqrt(a)
 [1.         1.41421356 1.73205081]

np.exp(a)
 [ 2.71828183  7.3890561  20.08553692]

np.log(a)
 [0.         0.69314718 1.09861229]

np.sin(a)
 [0.84147098 0.90929743 0.14112001]

np.cos(a)
 [ 0.54030231 -0.41614684 -0.9899925 ]

np.abs(a)
 [1 2 3]
```

[13]:
```
x = np.array([[1, 2, 3], [4, 5, 6]])
y = np.array([[7, 8, 9], [10, 11, 12]])

print("x : \n", x)
print("y : \n", y)
print()
```

```
x :
 [[1 2 3]
```

```
 [4 5 6]]
y :
 [[ 7  8  9]
 [10 11 12]]
```

[14]:
```
print("x.reshape(3, 2) \n", x.reshape(3, 2), "\n")
print("x.ravel() \n", x.ravel(), "\n")
print("x.flatten() \n", x.flatten(), "\n")
print("x.T \n", x.T, "\n")
print("np.concatenate(([x, y]), axis=0) \n", np.concatenate(([x, y]), axis=0),␣
 ↪"\n")
print("np.vstack(([x, y])) \n", np.vstack(([x, y])), "\n")
print("np.hstack(([x, y])) \n", np.hstack(([x, y])), "\n")
print("np.split(x, 2) \n", np.split(x, 2), "\n")
print("np.expand_dims(x, axis=0) \n", np.expand_dims(x, axis=0), "\n")
print("np.squeeze(np.expand_dims(x, axis=0)) \n", np.squeeze(np.expand_dims(x,␣
 ↪axis=0)), "\n")
```

```
x.reshape(3, 2)
 [[1 2]
 [3 4]
 [5 6]]

x.ravel()
 [1 2 3 4 5 6]

x.flatten()
 [1 2 3 4 5 6]

x.T
 [[1 4]
 [2 5]
 [3 6]]

np.concatenate(([x, y]), axis=0)
 [[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]

np.vstack(([x, y]))
 [[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]

np.hstack(([x, y]))
```

```
 [[ 1  2  3  7  8  9]
 [ 4  5  6 10 11 12]]

np.split(x, 2)
 [array([[1, 2, 3]]), array([[4, 5, 6]])]

np.expand_dims(x, axis=0)
 [[[1 2 3]
  [4 5 6]]]

np.squeeze(np.expand_dims(x, axis=0))
 [[1 2 3]
 [4 5 6]]
```

[15]:
```
print("x : ", x, "\n")
print("x.sum() \n", x.sum(), "\n")
print("np.sum(x, axis=0) \n", np.sum(x, axis=0), "\n")
print("x.mean() \n", x.mean(), "\n")
print("np.mean(x, axis=1) \n", np.mean(x, axis=1), "\n")
print("x.std() \n", x.std(), "\n")
print("x.var() \n", x.var(), "\n")
print("x.min() \n", x.min(), "\n")
print("x.max() \n", x.max(), "\n")
print("x.argmin() \n", x.argmin(), "\n")
print("x.argmax() \n", x.argmax(), "\n")
print("np.median(x) \n", np.median(x), "\n")
print("np.percentile(x, 75) \n", np.percentile(x, 75), "\n")
print("np.cumsum(x) \n", np.cumsum(x), "\n")
print("np.cumprod(x) \n", np.cumprod(x), "\n")
```

```
x :  [[1 2 3]
 [4 5 6]]

x.sum()
 21

np.sum(x, axis=0)
 [5 7 9]

x.mean()
 3.5

np.mean(x, axis=1)
 [2. 5.]

x.std()
 1.707825127659933
```

```
x.var()
 2.9166666666666665

x.min()
 1

x.max()
 6

x.argmin()
 0

x.argmax()
 5

np.median(x)
 3.5

np.percentile(x, 75)
 4.75

np.cumsum(x)
 [ 1  3  6 10 15 21]

np.cumprod(x)
 [  1   2   6  24 120 720]
```

[16]:
```
p = np.array([[4, 7], [2, 6]])
q = np.array([[1, 2], [3, 4]])

print("p : \n", p)
print("q : \n", q)
print()
```

```
p :
 [[4 7]
 [2 6]]
q :
 [[1 2]
 [3 4]]
```

[17]:
```
print("np.dot(p, q) \n", np.dot(p, q), "\n")
print("p @ q \n", p @ q, "\n")
print("np.linalg.inv(p) \n", np.linalg.inv(p), "\n")
print("np.linalg.det(p) \n", np.linalg.det(p), "\n")
```

```
print("np.linalg.eig(p) \n", np.linalg.eig(p), "\n")
print("np.linalg.norm(p) \n", np.linalg.norm(p), "\n")
print("np.linalg.solve(p, q) \n", np.linalg.solve(p, q), "\n")
```

```
np.dot(p, q)
 [[25 36]
 [20 28]]

p @ q
 [[25 36]
 [20 28]]

np.linalg.inv(p)
 [[ 0.6 -0.7]
 [-0.2  0.4]]

np.linalg.det(p)
 10.000000000000002

np.linalg.eig(p)
 EigResult(eigenvalues=array([1.12701665, 8.87298335]),
eigenvectors=array([[-0.92511345, -0.82071729],
       [ 0.37969079, -0.57133452]]))

np.linalg.norm(p)
 10.246950765959598

np.linalg.solve(p, q)
 [[-1.5 -1.6]
 [ 1.   1.2]]
```